(Multi-Agent) Reinforcement Learning: a primer, with applications in economics

Deep Reinforcement Learning

Aldo Glielmo
Applied Research Team, Banca d'Italia

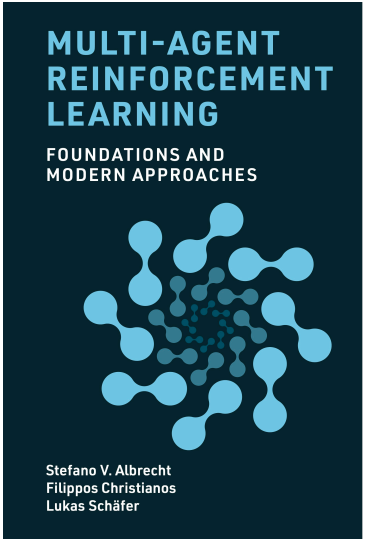
The MARL Book

A lot of material in this course is based on

Multi-Agent Reinforcement Learning: Foundations and Modern Approaches

The book is freely available from:

www.marl-book.com



Today's Lecture

Function approximation

- Motivation and deep learning for function approximation

Deep RL

- Deep-Q learning
- Moving target problem and correlations in consecutive experiences

  $\Rightarrow$ Short break (10 min, until around 11:30)
- Policy gradient theorem
- Actor-critic algorithms and Proximal Policy Optimization

  $\Rightarrow$ Longer break (15 min, until around 12:30)

Interactive session

Recap (L1): RL Basics

- **RL as sequential decision making:** learn a policy π by interaction to maximize

$$\mathbb{E}_\pi\left[\sum_{t=0}^{\infty}\gamma^t r^t\right].$$

- **MDP:** $(S, A, \mathcal{T}, \mathcal{R}, \gamma)$ and **value functions** V^π, Q^π with Bellman recursion.
- **Model-based DP (known $\mathcal{T}, \mathcal{R}$):** Policy Iteration / Value Iteration.
- **Model-free TD (from data):** SARSA (on-policy) vs Q-learning (off-policy) with ϵ-greedy exploration.

Interactive example $\rightarrow$ **https://awjuliani.github.io/web-rl-playground/**

Next: Deep RL $\Rightarrow$ approximate $V/Q/\pi$ with neural networks.

Function Approximation

Motivation for Function Approximation in RL

Problem

Previously introduced RL algorithms maintain **tabular** value functions with a value for each state-action pair.

Motivation for Function Approximation in RL

Problem

Previously introduced RL algorithms maintain **tabular** value functions with a value for each state-action pair. This has 2 major limitations:

1. The table grows linearly with the number of possible inputs

Motivation for Function Approximation in RL

Problem

Previously introduced RL algorithms maintain **tabular** value functions with a value for each state-action pair. This has 2 major limitations:

1. The table grows linearly with the number of possible inputs $\rightarrow$ difficult in complex environments (e.g., Go has approximately 10^{170} possible states)

Motivation for Function Approximation in RL

Problem

Previously introduced RL algorithms maintain **tabular** value functions with a value for each state-action pair. This has 2 major limitations:

1. The table grows linearly with the number of possible inputs $\rightarrow$ difficult in complex environments (e.g., Go has approximately 10^{170} possible states)
2. Each value is updated in isolation

Motivation for Function Approximation in RL

Problem

Previously introduced RL algorithms maintain **tabular** value functions with a value for each state-action pair. This has 2 major limitations:

1. The table grows linearly with the number of possible inputs $\rightarrow$ difficult in complex environments (e.g., Go has approximately 10^{170} possible states)
2. Each value is updated in isolation $\rightarrow$ agents need to visit each state-action pair to obtain accurate values

Motivation for Function Approximation in RL

Problem

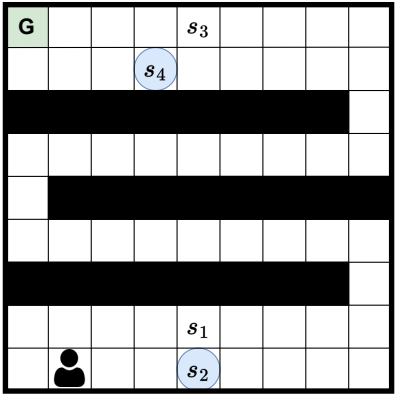
Previously introduced RL algorithms maintain **tabular** value functions with a value for each state-action pair. This has 2 major limitations:

1. The table grows linearly with the number of possible inputs $\rightarrow$ difficult in complex environments (e.g., Go has approximately 10^{170} possible states)
2. Each value is updated in isolation $\rightarrow$ agents need to visit each state-action pair to obtain accurate values

Solution

We need value functions that **generalise** across inputs.

Generalization



- The agent must reach the goal location (G)
- The agent has already seen states s_1 and s_3 during past trajectories, but not s_2 and s_4
- States s_2 and s_4 are similar to s_1 and s_3 respectively
- It would be beneficial if the agent could generalize from s_1 and s_3 to estimate the values of s_2 and s_4

Function Approximation

Function approximation is one way to find generalizable representations for value functions.

- Function approximations learns a **parameterized** function $f(x; \theta)$ for some input $x \in \mathbb{R}^{d_x}$ such that:

$$\forall x : f(x; \theta) \approx f^*(x)$$

- Training a function approximator involves optimizing the parameters θ such that f accurately **approximates** f^*
- In RL, f^* might be the true value function of the environment representing the expected return given a state s as input

Function Approximation Optimization

We might optimize $f(x; \theta)$ using a **mean squared error** loss.

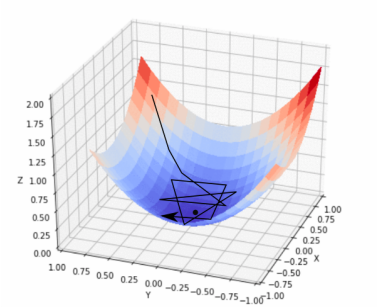
$$\theta^* = \arg\min_{\theta} \mathbb{E}_{s \in S} \left[(V^{\pi}(s) - \hat{V}(s; \theta))^2 \right]$$

Function Approximation Optimization

We might optimize $f(x;\theta)$ using a **mean squared error** loss.

$$\theta^* = \arg\min_{\theta} \mathbb{E}_{s\in S}\left[\left(V^{\pi}(s) - \hat{V}(s;\theta)\right)^2\right]$$

- We can find optimal θ values by computing gradients of all $\theta_i \in \theta$ with respect to the loss and descend along the gradient



Function Approximation Optimization

We might optimize $f(x;\theta)$ using a **mean squared error** loss.

$$\theta^* = \arg\min_{\theta} \mathbb{E}_{s \in S}\left[(V^{\pi}(s) - \hat{V}(s;\theta))^2\right]$$

- We can find optimal θ values by computing gradients of all $\theta_i \in \theta$ with respect to the loss and descend along the gradient
- With optimal parameters $\hat{V}(s;\theta^*)$ might be close to the true value function, allowing us to estimate values by 'plugging in' new potentially unseen states

Function Approximation Optimization

We might optimize $f(x;\theta)$ using a **mean squared error** loss.

$$\theta^* = \arg\min_{\theta} \mathbb{E}_{s\in S}\left[(V^{\pi}(s) - \hat{V}(s;\theta))^2\right]$$

- We can find optimal θ values by computing gradients of all $\theta_i \in \theta$ with respect to the loss and descend along the gradient
- With optimal parameters $\hat{V}(s;\theta^*)$ might be close to the true value function, allowing us to estimate values by 'plugging in' new potentially unseen states
- Linear function approximation is a simple and powerful way to find generalizable functions

Deep Learning for Function Approximation

The main benefit of linear value functions is their simplicity and generalization ability. However, linear function approximation heavily relies on the selection of state features.

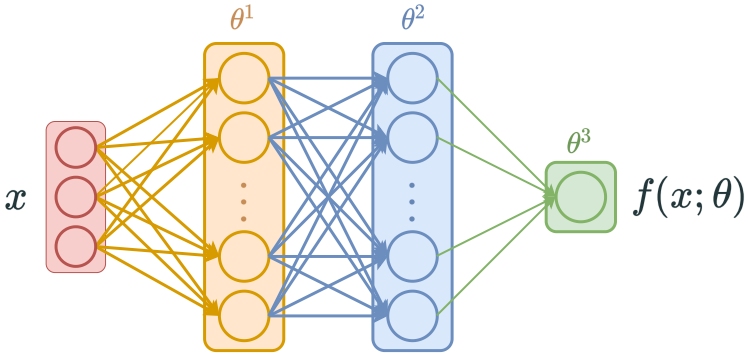
Deep Learning Advantages:

- Universal method, leveraging **neural networks** for function approximation
- Automatically learns feature representations of states
- Able to represent non-linear, complex functions
- Is effective with high-dimensional data such as images and text

Neural Networks

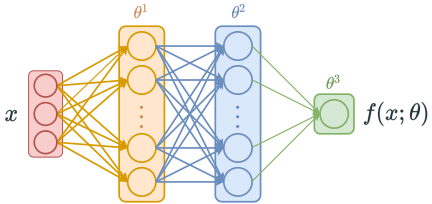
Feedforward Neural Networks

Feedforward neural networks, fully-connected neural networks, or multi-layer perceptrons can be considered as the building block of most neural networks.



Feedforward Neural Networks

Feedforward neural networks, fully-connected neural networks, or multi-layer perceptrons can be considered as the building block of most neural networks.



- A feedforward NN consists of multiple sequential layers
- The first layer processes input x, subsequent layers process the outputs of previous layers
- Each layer is defined as a parameterized function and is composed of many neural units
- We typically refer to the first layer as the **input layer** and the last layer as the **output layer** and all layers in between as **hidden layers**

Universal Function Approximation Theory

The **universal function approximation theory** states that with as little as one hidden layer, a feedforward neural network can approximate **any** function.

- This theory assumes arbitrary **width** of the hidden layer
- We refer to the number of neural units within hidden layers as the **width** of the network
- The number of layers is referred to as the **depth**
- In practice, **deep** neural networks can outperform shallow ones and in some cases, require fewer parameters to do so

Convolution and Recurrent Neural Networks

Feedforward neural networks can, in principle, be applied to any data. There are, however, architectures designed to perform well with specific data.

Architectures often used in RL are:

1. **Convolutional neural networks**
   - Used to process images
   - Aims to make use of spatial correlations
   - Often incorporate inductive biases such as invariance to local image-based translations
2. **Recurrent neural networks**
   - Used to process temporal data
   - Makes use of temporal correlations

Convolutional Neural Networks

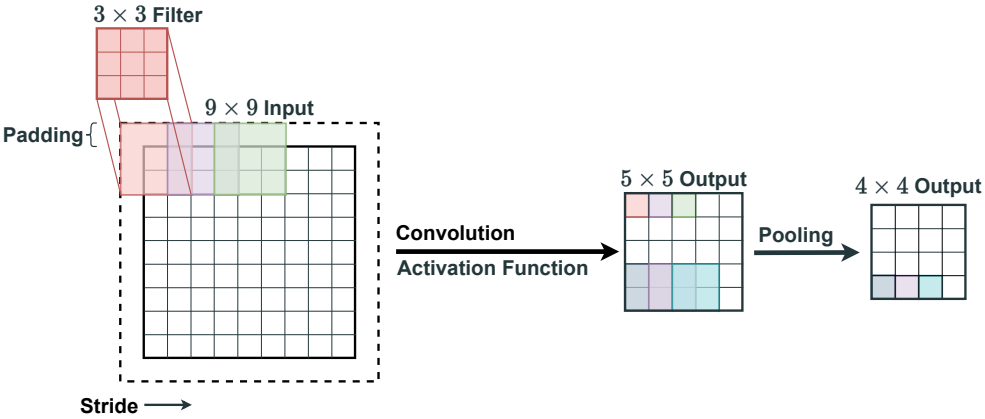
Convolutional neural networks (CNNs) are neural networks for image processing

- Blocks of parameters (called **filters** or **kernels**) are moved and repeatedly applied across an image

Some advantages to this architecture include:

- **Parameter sharing:** kernel significantly reduces the total number of parameters compared to a feedforward NN
- For example to process a 128×128 RGB image:
  - Feedforward NN: input layer with 128 hidden units has 6,291,584 parameters
  - CNN: with sixteen 5×5 filters has 1,216 parameters
- **Spatial correlations:** nearby pixels are likely related (e.g., edges) $\rightarrow$ same spatial patterns may occur at different locations of an image

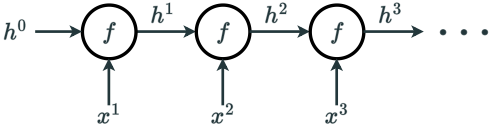
Convolutional Neural Networks



Recurrent Neural Networks

Recurrent neural networks (RNNs) are designed to process **sequential** data with several advantages:

- Learning sequences may require passing in the entire sequence as an input, which requires lots of parameters
- It is also reasonable to assume a correlation between time points in a sequence
- Sharing parameters might help pick up on recurring patterns in the data that occur at different timesteps

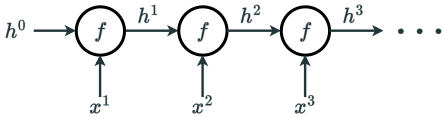


Recurrent Neural Networks

RNNs use a **hidden state**, passed as an additional input.

- RNNs sequentially process inputs (i.e., first x^1 then x^2 ...)
- The RNN also conditions on the **hidden state** h^t
- h^t can be thought of as a compact representation of the entire history of past inputs

Function: $h^t = f(x^t, h^{t-1}; \theta)$



Deep Q-Learning

Recap: Q-Learning

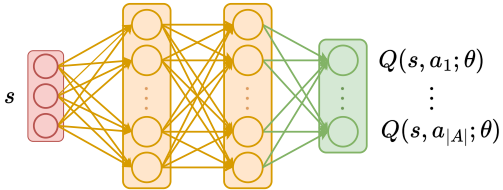
Reminder

The Bellman update rule in tabular Q-learning is:

$$\underbrace{Q(s^t, a^t)}_{\text{New Q-value}} \leftarrow \underbrace{Q(s^t, a^t)}_{\text{Old Q-value}} + \underbrace{\alpha}_{\text{Learning rate}} \underbrace{\left(\underbrace{\underbrace{r^t}_{\text{Reward}} + \underbrace{\gamma \max_{a'} Q(s^{t+1}, a')}_{\text{Discounted optimal Q-value for next state}}}_{\text{TD target}} - \underbrace{Q(s^t, a^t)}_{\text{Old Q-value}} \right)}_{\text{TD error}}$$

Deep Q-Learning

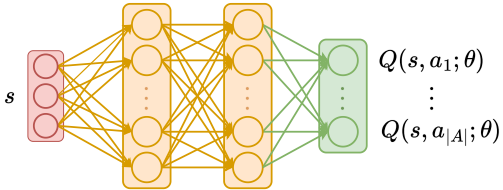
For **deep** Q-learning, we use a neural network to approximate the Q function.



- To train this we could define a loss function $\mathcal{L}(\theta) = (y^t - Q(s^t, a^t; \theta))^2$
- But unlike supervised learning, we are not given y^t beforehand

Deep Q-Learning

For **deep** Q-learning, we use a neural network to approximate the Q function.



- To train this we could define a loss function $\mathcal{L}(\theta) = (y^t - Q(s^t, a^t; \theta))^2$
- But unlike supervised learning, we are not given y^t beforehand
- We can use the Q-learning update rule to define our y^t

$$y^t = \begin{cases} r^t & \text{if } s^{t+1} \text{ is terminal} \\ r^t + \gamma \max_{a'} Q(s^{t+1}, a'; \theta) & \text{otherwise} \end{cases}$$

Naive Deep Q-Learning Pseudo-Code

Algorithm Deep Q-learning

```
Initialize value network Q with random parameters θ
for every episode do
    for time step t = 0, 1, 2, ... do
        Observe current state s^t
        With probability ϵ: choose random action a^t ∈ A
        Otherwise: choose a^t ∈ arg max_a Q(s^t, a; θ)
        Apply action a^t; observe reward r^t and next state s^{t+1}
        if s^{t+1} is terminal then
            Target y^t ← r^t
        else
            Target y^t ← r^t + γ max_{a'} Q(s^{t+1}, a'; θ)
        Loss L(θ) ← (y^t − Q(s^t, a^t; θ))^2
        Update parameters θ by minimising the loss L(θ)
```

Naive Deep Q-Learning Pseudo-Code

Algorithm Deep Q-learning

```
Initialize value network Q with random parameters θ
for every episode do
    for time step t = 0, 1, 2, ... do
        Observe current state s^t
        With probability ϵ: choose random action a^t ∈ A
        Otherwise: choose a^t ∈ arg max_a Q(s^t, a; θ)
        Apply action a^t; observe reward r^t and next state s^{t+1}
        if s^{t+1} is terminal then
            Target y^t ← r^t
        else
            Target y^t ← r^t + γ max_{a'} Q(s^{t+1}, a'; θ)
        Loss L(θ) ← (y^t − Q(s^t, a^t; θ))^2
        Update parameters θ by minimising the loss L(θ)
```

This naive application of neural networks to RL algorithms has some problems.

The Moving Target Problem

Problem

Moving target problem arises from the bootstrapped targets:

$$y^t = r^t + \gamma \max_{a'} Q(s^{t+1}, a'; \theta)$$

The Moving Target Problem

Problem

Moving target problem arises from the bootstrapped targets:

$$y^t = r^t + \gamma \max_{a'} Q(s^{t+1}, a'; \theta)$$

- Value function with NNs generalize value estimates across inputs
- As targets y^t depend on θ, any update to θ changes the target

  $\Rightarrow$ This non-stationarity of the targets makes it difficult to learn optimal θ

The Moving Target Problem

Problem

Moving target problem arises from the bootstrapped targets:

$$y^t = r^t + \gamma \max_{a'} Q(s^{t+1}, a'; \theta)$$

- Value function with NNs generalize value estimates across inputs
- As targets y^t depend on θ, any update to θ changes the target

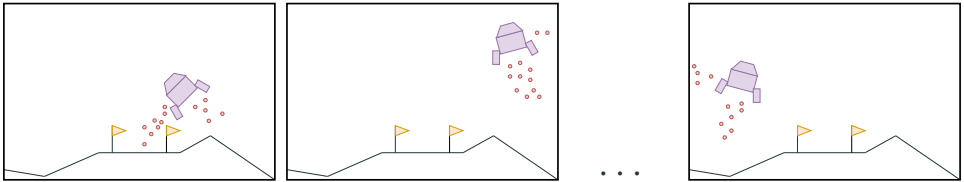
  $\Rightarrow$ This non-stationarity of the targets makes it difficult to learn optimal θ

Solution

One solution is to use a **target network** with parameters $\bar{\theta}$ that are updated less often than our Q network's parameters θ

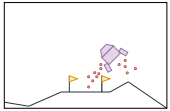
Correlation of Consecutive Experiences

- Most ML algorithms using NNs assume i.i.d. data
- In RL, we collect data by interacting with an MDP with $s^{t+1} \sim \mathcal{T}(s^t, a^t) \rightarrow$ **not** i.i.d

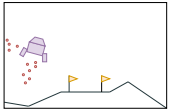


Correlation of Consecutive Experiences

- Most ML algorithms using NNs assume i.i.d. data
- In RL, we collect data by interacting with an MDP with $s^{t+1} \sim \mathcal{T}(s^t, a^t) \rightarrow$ **not** i.i.d



. . .



Problem

This correlated data can lead to **overfitting** of the value function to recent experiences, and result in **catastophic forgetting** of previously learned estimates.

Correlation of Consecutive Experiences

- Most ML algorithms using NNs assume i.i.d. data
- In RL, we collect data by interacting with an MDP with $s^{t+1} \sim \mathcal{T}(s^t, a^t) \rightarrow$ **not** i.i.d

Problem

This correlated data can lead to **overfitting** of the value function to recent experiences, and result in **catastophic forgetting** of previously learned estimates.

Solution

We train on samples of previous experiences stored in a **replay buffer**.

Deep Q-Networks

Deep Q-networks (DQN) is a foundational deep RL algorithm based on Q-learning.

- Tabular value function $\longrightarrow$ neural network
- Moving target problem $\longrightarrow$ target networks
- Correlated experiences $\longrightarrow$ replay buffer

Deep Q-Networks

Deep Q-networks (DQN) is a foundational deep RL algorithm based on Q-learning.

Target networks:

- Compute target estimates with target network parameters $\bar{\theta}$:

$$y^t \leftarrow r^t + \gamma \max_{a'} Q(s^{t+1}, a'; \bar{\theta})$$

Deep Q-Networks

Deep Q-networks (DQN) is a foundational deep RL algorithm based on Q-learning.

Target networks:

- Compute target estimates with target network parameters $\bar{\theta}$:

$$y^t \leftarrow r^t + \gamma \max_{a'} Q(s^{t+1}, a'; \bar{\theta})$$

- Select actions according to the "main" value network $\rightarrow a^t \in \arg\max_a Q(s^t, a; \theta)$

Deep Q-Networks

Deep Q-networks (DQN) is a foundational deep RL algorithm based on Q-learning.

Target networks:

- Compute target estimates with target network parameters $\bar{\theta}$:

$$y^t \leftarrow r^t + \gamma \max_{a'} Q(s^{t+1}, a'; \bar{\theta})$$

- Select actions according to the "main" value network $\rightarrow a^t \in \arg\max_a Q(s^t, a; \theta)$
- Update the "main" value network parameters θ by minimizing the DQL loss $\mathcal{L}(\theta)$

Deep Q-Networks

Deep Q-networks (DQN) is a foundational deep RL algorithm based on Q-learning.

Target networks:

- Compute target estimates with target network parameters $\bar{\theta}$:

$$y^t \leftarrow r^t + \gamma \max_{a'} Q(s^{t+1}, a'; \bar{\theta})$$

- Select actions according to the "main" value network $\rightarrow a^t \in \arg\max_a Q(s^t, a; \theta)$
- Update the "main" value network parameters θ by minimizing the DQL loss $\mathcal{L}(\theta)$
- Update the **target network** parameters in regular intervals $\bar{\theta} \leftarrow \theta$

Deep Q-Networks

Deep Q-networks (DQN) is a foundational deep RL algorithm based on Q-learning.

Replay buffers:

- Store experience tuples (s^t, a^t, r^t, s^{t+1}) in a replay buffer $\mathcal{D}$

Deep Q-Networks

Deep Q-networks (DQN) is a foundational deep RL algorithm based on Q-learning.

Replay buffers:

- Store experience tuples (s^t, a^t, r^t, s^{t+1}) in a replay buffer $\mathcal{D}$
- To compute the loss, sample batches of experience tuples (uniformly at random) from the replay buffer $\mathcal{B} \sim \mathcal{U}(\mathcal{D})$

Deep Q-Networks

Deep Q-networks (DQN) is a foundational deep RL algorithm based on Q-learning.

Replay buffers:

- Store experience tuples (s^t, a^t, r^t, s^{t+1}) in a replay buffer $\mathcal{D}$
- To compute the loss, sample batches of experience tuples (uniformly at random) from the replay buffer $\mathcal{B} \sim \mathcal{U}(\mathcal{D})$
- Random sampling "breaks" correlations, allowing for a more stable optimization

Deep Q-Networks

Deep Q-networks (DQN) is a foundational deep RL algorithm based on Q-learning.

Replay buffers:

- Store experience tuples (s^t, a^t, r^t, s^{t+1}) in a replay buffer $\mathcal{D}$
- To compute the loss, sample batches of experience tuples (uniformly at random) from the replay buffer $\mathcal{B} \sim \mathcal{U}(\mathcal{D})$
- Random sampling "breaks" correlations, allowing for a more stable optimization
- Also reuses experiences during training $\rightarrow$ improved sample efficiency

Deep Q-Networks

Deep Q-networks (DQN) is a foundational deep RL algorithm based on Q-learning.

Replay buffers:

- Store experience tuples (s^t, a^t, r^t, s^{t+1}) in a replay buffer $\mathcal{D}$
- To compute the loss, sample batches of experience tuples (uniformly at random) from the replay buffer $\mathcal{B} \sim \mathcal{U}(\mathcal{D})$
- Random sampling "breaks" correlations, allowing for a more stable optimization
- Also reuses experiences during training $\rightarrow$ improved sample efficiency

Note

Replay buffers can only be used for **off-policy** algorithms. We use experiences collected from previous (different) policies, which change as we update θ.

DQN Pseudo Code

Algorithm Deep Q-networks (DQN)

```
Initialize value network Q with random parameters θ
Initialize target network with parameters θ̄ = θ
Initialize an empty replay buffer D = {}
for every episode do
    for time step t = 0, 1, 2, ... do
        Observe current state s^t
        With probability ε: choose random action a^t ∈ A
        Otherwise: choose a^t ∈ arg max_a Q(s^t, a; θ)
        Apply action a^t; observe reward r^t and next state s^{t+1}
        Store transition (s^t, a^t, r^t, s^{t+1}) in replay buffer D
        Sample random mini-batch of B transitions (s^k, a^k, r^k, s^{k+1}) from D
        if s^{k+1} is terminal then
            Targets y^k ← r^k
        else
            Targets y^k ← r^k + γ max_{a'} Q(s^{k+1}, a'; θ̄)
        Loss L(θ) ← (1/B) Σ_{k=1}^{B} (y^k − Q(s^k, a^k; θ))^2
        Update parameters θ by minimising the loss L(θ)
        In a set interval, update target network parameters θ̄
```

Overestimation Bias

Problem

DQN tends to **overestimate** values with 1-step targets

$$y^k \leftarrow r^k + \gamma \max_{a'} Q(s^{k+1}, a'; \bar{\theta})$$

- Using the max operator, select the maximum value estimate for the target
- Since our value estimates do not necessarily reflect the true value function, the max operation will likely select an overestimated action-value estimate
- This can slow down the convergence of the algorithm as the agent spends too much time exploring states with overestimated values

Double Q-Learning

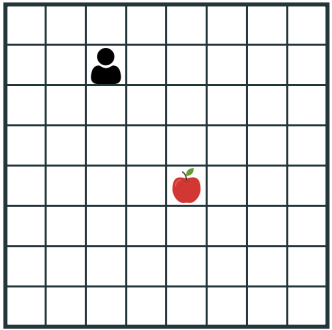
Solution

Double Q-learning reduces this overestimation bias by decoupling the action selection from value estimation using separate function approximations.

- This can be achieved with minimal changes in DQN
- DDQN uses the primary Q-network (with parameters θ) to select actions while using the target network (with parameters $\bar{\theta}$) to estimate action values
- The target thus becomes:

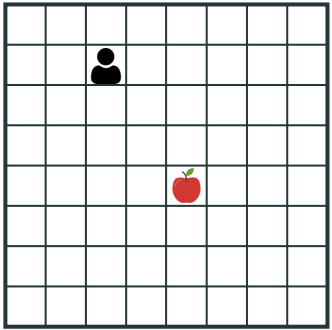
$$y^t = \begin{cases} r^t & \text{if } s^{t+1} \text{ is terminal} \\ r^t + \gamma Q(s^{t+1}, \arg\max_{a'} Q(s^{t+1}, a'; \theta); \bar{\theta}) & \text{otherwise} \end{cases}$$

Deep Q-learning in Practice

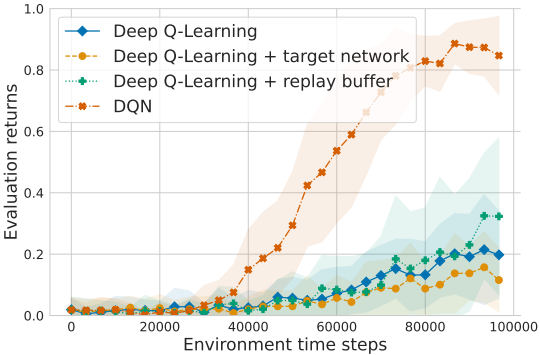


(a) Single-agent level-based foraging environment

Deep Q-learning in Practice

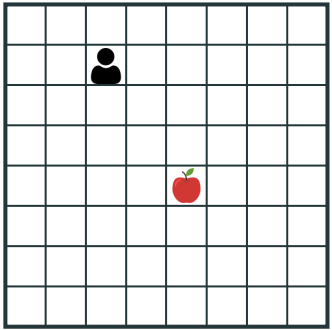


(a) Single-agent level-based foraging environment

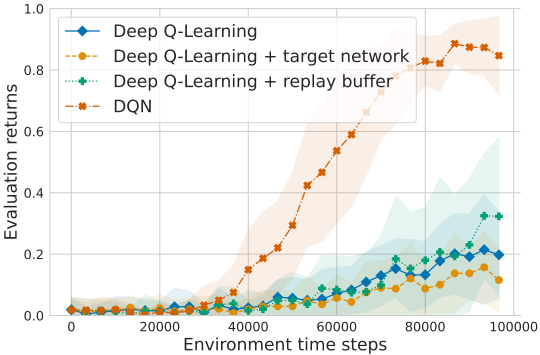


(b) Learning curves

Deep Q-learning in Practice



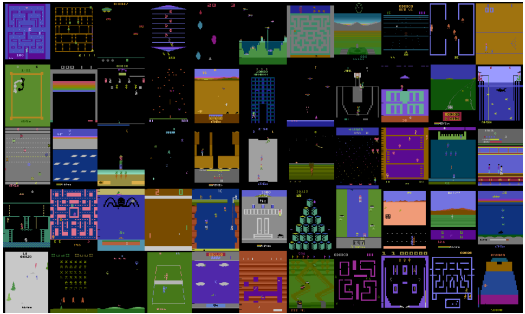
(a) Single-agent level-based foraging environment



(b) Learning curves

Note that in isolation, neither the addition of **target networks** nor of a **replay buffer** are sufficient to stably train the agent with deep Q-learning in this environment.

DQN for Atari 2600 (Arcade Learning Environment)



Atari 2600 (Arcade Learn. Env.)

- Human-level performance from pixels
- Single CNN architecture across games
- Stabilization via experience replay and target networks
- DeepMind, 2015, Nature
- Example video: **link**

DQN – Interactive Visualisation

- Colab notebook **link**
- Some things to check:

  $\Rightarrow$ How does the loss evolve during training?

  $\Rightarrow$ Do you understand the different hyperparameters and their effect on the learning process?

Policy Gradient Algorithms

Policy Gradients

We considered a parameterized value function, but we can also directly parameterize the policy π.

- Use a NN for policy π with parameters ϕ
- Policy network receives state s as input and outputs a scalar value for each action
- Scalars $l(s, a)$ represent the preference of the policy to select action a in state s

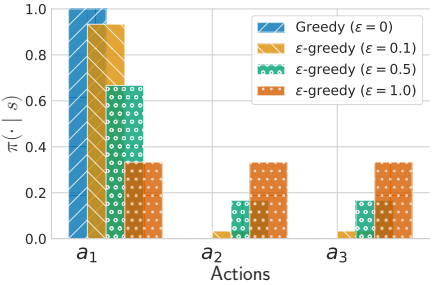
Policy Gradients

We considered a parameterized value function, but we can also directly parameterize the policy π.

- Use a NN for policy π with parameters ϕ
- Policy network receives state s as input and outputs a scalar value for each action
- Scalars $l(s, a)$ represent the preference of the policy to select action a in state s
- Preferences are then transformed into a probability distribution across the action space using a **softmax** function:

$$\pi(a \mid s; \phi) = \frac{e^{l(s,a;\phi)}}{\sum_{a' \in A} e^{l(s,a';\phi)}}$$

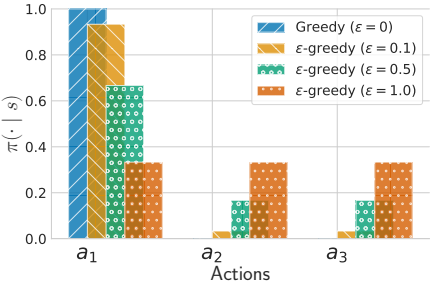
Advantages of Learning a Policy



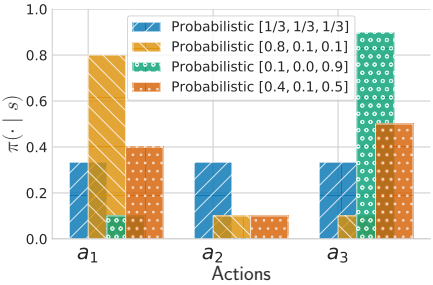
ϵ-greedy policies

- ϵ-greedy policies struggle to represent diverse probabilistic policies

Advantages of Learning a Policy



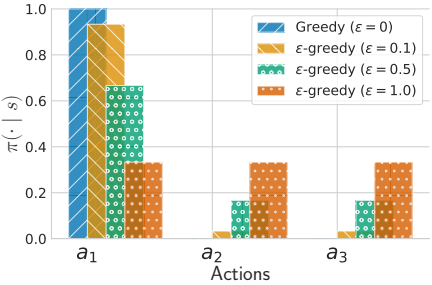
ϵ-greedy policies



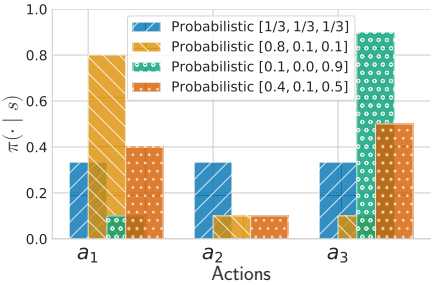
Probabilistic policies

- ϵ-greedy policies struggle to represent diverse probabilistic policies
- Policy gradient algorithms allow us to represent **any** probabilistic policy

Advantages of Learning a Policy



ϵ-greedy policies



Probabilistic policies

- ϵ-greedy policies struggle to represent diverse probabilistic policies
- Policy gradient algorithms allow us to represent **any** probabilistic policy
- Policy gradients are also effective for representing **continuous** action spaces

Policy Gradient Theorem

How to update parameters ϕ of the policy? Using the **policy gradient theorem**, we can express the gradient of the performance of a policy with respect to the parameter ϕ of the policy.

$$\nabla_\phi J(\phi) \propto \sum_{s \in S} \Pr(s \mid \pi) \sum_{a \in A} Q^\pi(s, a) \nabla_\phi \pi(a \mid s; \phi)$$

- J is a function measuring the quality of policy π
- $Pr(s \mid \pi)$ is the state-visitation distribution for policy π
- $Q^\pi(s, a)$ is the value for a given action and state under π
- The J function is similar to a loss function, with the difference that we aim to **maximize** rather than minimize it

Policy Gradient Theorem

How to update parameters ϕ of the policy? Using the **policy gradient theorem**, we can express the gradient of the performance of a policy with respect to the parameter ϕ of the policy.

$$\nabla_\phi J(\phi) \propto \sum_{s \in S} \Pr(s \mid \pi) \sum_{a \in A} Q^\pi(s, a) \nabla_\phi \pi(a \mid s; \phi)$$

Note

The policy gradient theorem assumes that $Pr(s \mid \pi)$ are given under the currently optimized policy $\pi \rightarrow$ this needs **on-policy** data

Policy Gradient Theorem – Continued

The assumption that $Pr(s \mid \pi)$ is **on-policy** becomes apparent through a simple derivation.

$$\nabla_\phi J(\phi) \propto \sum_{s \in S} \Pr(s \mid \pi) \sum_{a \in A} Q^\pi(s, a) \nabla_\phi \pi(a \mid s; \phi)$$

Policy Gradient Theorem – Continued

The assumption that $Pr(s \mid \pi)$ is **on-policy** becomes apparent through a simple derivation.

$$\begin{aligned}\nabla_\phi J(\phi) &\propto \sum_{s \in S} \Pr(s \mid \pi) \sum_{a \in A} Q^\pi(s, a) \nabla_\phi \pi(a \mid s; \phi) \\ &= \mathbb{E}_{s \sim \Pr(\cdot \mid \pi)} \left[\sum_{a \in A} Q^\pi(s, a) \nabla_\phi \pi(a \mid s; \phi) \right]\end{aligned}$$

Policy Gradient Theorem – Continued

The assumption that $Pr(s \mid \pi)$ is **on-policy** becomes apparent through a simple derivation.

$$
\begin{aligned}
\nabla_\phi J(\phi) &\propto \sum_{s \in S} \Pr(s \mid \pi) \sum_{a \in A} Q^\pi(s, a) \nabla_\phi \pi(a \mid s; \phi) \\
&= \mathbb{E}_{s \sim \Pr(\cdot \mid \pi)} \left[\sum_{a \in A} Q^\pi(s, a) \nabla_\phi \pi(a \mid s; \phi) \right] \\
&= \mathbb{E}_{s \sim \Pr(\cdot \mid \pi)} \left[\sum_{a \in A} \pi(a \mid s; \phi) Q^\pi(s, a) \frac{\nabla_\phi \pi(a \mid s; \phi)}{\pi(a \mid s; \phi)} \right]
\end{aligned}
$$

Policy Gradient Theorem – Continued

The assumption that $Pr(s \mid \pi)$ is **on-policy** becomes apparent through a simple derivation.

$$
\begin{aligned}
\nabla_\phi J(\phi) &\propto \sum_{s \in S} \Pr(s \mid \pi) \sum_{a \in A} Q^\pi(s, a) \nabla_\phi \pi(a \mid s; \phi) \\
&= \mathbb{E}_{s \sim \Pr(\cdot \mid \pi)} \left[\sum_{a \in A} Q^\pi(s, a) \nabla_\phi \pi(a \mid s; \phi) \right] \\
&= \mathbb{E}_{s \sim \Pr(\cdot \mid \pi)} \left[\sum_{a \in A} \pi(a \mid s; \phi) Q^\pi(s, a) \frac{\nabla_\phi \pi(a \mid s; \phi)}{\pi(a \mid s; \phi)} \right] \\
&= \mathbb{E}_{s \sim \Pr(\cdot \mid \pi), a \sim \pi(\cdot \mid s; \phi)} \left[Q^\pi(s, a) \frac{\nabla_\phi \pi(a \mid s; \phi)}{\pi(a \mid s; \phi)} \right]
\end{aligned}
$$

Policy Gradient Theorem – Continued

The assumption that $Pr(s \mid \pi)$ is **on-policy** becomes apparent through a simple derivation.

$$
\begin{aligned}
\nabla_\phi J(\phi) &\propto \sum_{s \in S} \Pr(s \mid \pi) \sum_{a \in A} Q^\pi(s, a) \nabla_\phi \pi(a \mid s; \phi) \\
&= \mathbb{E}_{s \sim \Pr(\cdot \mid \pi)} \left[\sum_{a \in A} Q^\pi(s, a) \nabla_\phi \pi(a \mid s; \phi) \right] \\
&= \mathbb{E}_{s \sim \Pr(\cdot \mid \pi)} \left[\sum_{a \in A} \pi(a \mid s; \phi) Q^\pi(s, a) \frac{\nabla_\phi \pi(a \mid s; \phi)}{\pi(a \mid s; \phi)} \right] \\
&= \mathbb{E}_{s \sim \Pr(\cdot \mid \pi), a \sim \pi(\cdot \mid s; \phi)} \left[Q^\pi(s, a) \frac{\nabla_\phi \pi(a \mid s; \phi)}{\pi(a \mid s; \phi)} \right] \\
&= \mathbb{E}_{s \sim \Pr(\cdot \mid \pi), a \sim \pi(\cdot \mid s; \phi)} \left[Q^\pi(s, a) \nabla_\phi \log \pi(a \mid s; \phi) \right]
\end{aligned}
$$

Policy Gradient Theorem – Continued

Intuition

$$\nabla_\phi J(\phi) \propto \mathbb{E}_{a\sim\pi(\cdot|s;\phi)}\left[Q^\pi(s,a)\frac{\nabla_\phi \pi(a \mid s;\phi)}{\pi(a \mid s;\phi)}\right]$$

We can interpret the components of the policy gradient theorem as follows:

- $\nabla_\phi \pi(a \mid s;\phi)$: gradient of ϕ pointing in the direction that most increase the probability of taking action a when in state s

Policy Gradient Theorem – Continued

Intuition

$$\nabla_\phi J(\phi) \propto \mathbb{E}_{a\sim\pi(\cdot|s;\phi)}\left[Q^\pi(s,a)\frac{\nabla_\phi \pi(a \mid s;\phi)}{\pi(a \mid s;\phi)}\right]$$

We can interpret the components of the policy gradient theorem as follows:

- $\nabla_\phi \pi(a \mid s;\phi)$: gradient of ϕ pointing in the direction that most increase the probability of taking action a when in state s
- $Q^\pi(s,a)$: expected returns as a quality measure of taking action a in state s

Policy Gradient Theorem – Continued

Intuition

$$\nabla_\phi J(\phi) \propto \mathbb{E}_{a\sim\pi(\cdot|s;\phi)}\left[Q^\pi(s,a)\frac{\nabla_\phi \pi(a \mid s;\phi)}{\pi(a \mid s;\phi)}\right]$$

We can interpret the components of the policy gradient theorem as follows:

- $\nabla_\phi \pi(a \mid s;\phi)$: gradient of ϕ pointing in the direction that most increase the probability of taking action a when in state s
- $Q^\pi(s,a)$: expected returns as a quality measure of taking action a in state s
- $\frac{1}{\pi(a|s;\phi)}$: normalization coefficient to account for varying probabilities of different actions under the policy

REINFORCE: Monte Carlo

To apply the policy gradient theorem, we must find a way to derive expected returns.

- **REINFORCE** is a policy gradient algorithm that uses Monte Carlo (MC) to estimate the expected returns of a policy

REINFORCE: Monte Carlo

To apply the policy gradient theorem, we must find a way to derive expected returns.

- **REINFORCE** is a policy gradient algorithm that uses Monte Carlo (MC) to estimate the expected returns of a policy
- The algorithm minimizes the following loss for an episodic history $h = \{s^0, a^0, r^0, ..., s^{T-1}, a^{T-1}, r^{T-1}, s^T\}$:

$$\mathcal{L}(\phi) = -\frac{1}{T}\sum_{t=0}^{T-1}\left(\sum_{\tau=t}^{T-1}\gamma^{\tau-t}r^{\tau}\right)\log\pi(a^t \mid s^t;\phi)$$

REINFORCE: Monte Carlo

To apply the policy gradient theorem, we must find a way to derive expected returns.

- **REINFORCE** is a policy gradient algorithm that uses Monte Carlo (MC) to estimate the expected returns of a policy
- The algorithm minimizes the following loss for an episodic history $h = \{s^0, a^0, r^0, ..., s^{T-1}, a^{T-1}, r^{T-1}, s^T\}$:

$$\mathcal{L}(\phi) = -\frac{1}{T}\sum_{t=0}^{T-1}\left(\sum_{\tau=t}^{T-1}\gamma^{\tau-t}r^{\tau}\right)\log\pi(a^t \mid s^t;\phi)$$

- Note this loss has a negative sign, as we want to **maximize** expected returns

REINFORCE Pseudocode

Algorithm REINFORCE

```
Initialize policy network π with random parameters φ
for every episode do
    for time step t = 0, 1, 2, ..., T − 1 do
        Observe current state s^t
        Sample action a^t ∼ π(· | s^t; φ)
        Apply action a^t; observe reward r^t and next state s^{t+1}
    Loss L(φ) ← −(1/T) Σ_{t=0}^{T−1} ( Σ_{τ=t}^{T−1} γ^{τ−t} r^τ ) log π(a^t | s^t; φ)
    Update parameters φ by minimizing the loss L(φ)
```

Baseline to Reduce Variance

Problem

High variance of MC estimates causes unstable gradients and training.

Baseline to Reduce Variance

Problem

High variance of MC estimates causes unstable gradients and training.

Solution

- Reduce variance by subtracting a **baseline** from the return estimates
- Common choice of baseline is state-value function $V(s)$ which can be trained to minimize the loss $\mathcal{L}(\theta) = \frac{1}{T}\sum_{t=1}^{T-1}(u(h^t) - V(s^t;\theta))^2$
- The REINFORCE policy loss then becomes

$$\mathcal{L}(\phi) = -\frac{1}{T}\sum_{t=0}^{T-1}\big(u(h^t) - V(s^t;\theta)\big)\log\pi(a^t \mid s^t;\phi)$$

Actor-Critic Algorithms

The policy gradient theorem allows us to optimize our parameterized policy; we still need to approximate expected returns.

- MC methods have high variance and require an entire episode to compute estimates
- **Actor-critic** algorithms aim to reduce variance and update more often by using **bootstrapped return estimates**

$$\begin{aligned}\mathbb{E}_\pi\left[u(h^t) \mid s^t\right] &= \mathbb{E}_\pi\left[R(s^t, a^t, s^{t+1}) + \gamma u(h^{t+1}) \mid s^t, a^t \sim \pi(\cdot \mid s^t)\right] \\ &= \mathbb{E}_\pi\left[R(s^t, a^t, s^{t+1}) + \gamma V(s^{t+1}) \mid s^t, a^t \sim \pi(\cdot \mid s^t)\right]\end{aligned}$$

- We now train a **critic** (value function approximator) alongside the **actor** (parameterized policy) which acts as a baseline

Balancing Bias and Variance

Problem

MC estimates have high **variance** and bootstrapping introduces **bias** since the value function might not yet approximate the true expected returns.

Balancing Bias and Variance

Problem

MC estimates have high **variance** and bootstrapping introduces **bias** since the value function might not yet approximate the true expected returns.

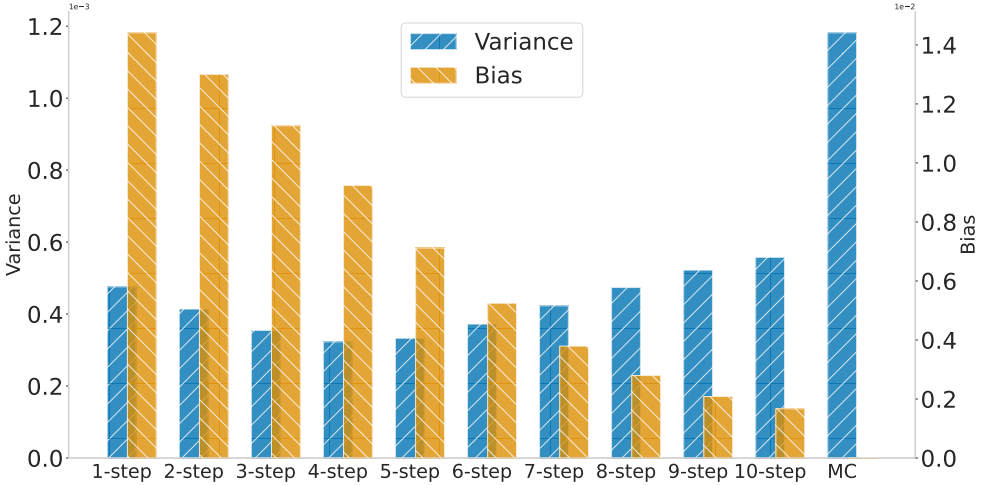
Solution

N-step returns allow us to balance between bias and variance:

$$\mathbb{E}_\pi\left[u(h^t) \mid s^t\right] = \mathbb{E}_\pi\left[\left(\sum_{\tau=0}^{N-1} \gamma^\tau \mathcal{R}(s^{t+\tau}, a^{t+\tau}, s^{t+\tau+1})\right) + \gamma^N V(s^{t+N}) \middle| s^t, a^\tau \sim \pi(\cdot \mid s^\tau)\right]$$

$N = 1 \rightarrow$ one-step bootstrapped returns $\ldots$ $N = T \rightarrow$ MC returns

Balancing Bias and Variance – Continued



Advantage Actor Critic (A2C)

Advantage actor-critic is a foundation actor-critic algorithm which uses the **advantage** of a policy to guide the policy gradients:

$$Adv^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

Advantage Actor Critic (A2C)

Advantage actor-critic is a foundation actor-critic algorithm which uses the **advantage** of a policy to guide the policy gradients:

$$Adv^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

We can compute the advantage using only a state-value function:

$$Adv(s^t, a^t) = Q(s^t, a^t) - V(s^t) = \begin{cases} r^t - V(s^t) & \text{if } s^{t+1} \text{ is terminal} \\ r^t + \gamma V(s^{t+1}) - V(s^t) & \text{otherwise} \end{cases}$$

Advantage Actor Critic (A2C)

Advantage actor-critic is a foundation actor-critic algorithm which uses the **advantage** of a policy to guide the policy gradients:

$$Adv^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

We can compute the advantage using only a state-value function:

$$Adv(s^t, a^t) = Q(s^t, a^t) - V(s^t) = \begin{cases} r^t - V(s^t) & \text{if } s^{t+1} \text{ is terminal} \\ r^t + \gamma V(s^{t+1}) - V(s^t) & \text{otherwise} \end{cases}$$

- We can also still use N-step returns to reduce bias during the advantage computation

A2C Pseudocode

Algorithm A2C

```
Initialize actor network π with random parameters ϕ
Initialize critic network V with random parameters θ
for every episode do
    for time step t = 0, 1, 2, ... do
        Observe current state s^t
        Sample action a^t ~ π(· | s^t; ϕ)
        Apply action a^t; observe reward r^t and next state s^{t+1}
        if s^{t+1} is terminal then
            Advantage Adv(s^t, a^t) ← r^t − V(s^t; θ)
            Critic target y^t ← r^t
        else
            Advantage Adv(s^t, a^t) ← r^t + γV(s^{t+1}; θ) − V(s^t; θ)
            Critic target y^t ← r^t + γV(s^{t+1}; θ)
        Actor loss L(ϕ) ← −Adv(s^t, a^t) log π(a^t | s^t; ϕ)
        Critic loss L(θ) ← (y^t − V(s^t; θ))^2
        Update parameters ϕ by minimizing the actor loss L(ϕ)
        Update parameters θ by minimizing the critic loss L(θ)
```

Proximal Policy Optimization (PPO)

Problem

Policy gradient methods can cause significant shifts in the policy with a single update, which can worsen the policy!

Solution

Limit the change of the policy in a single update $\rightarrow$ **trust region** of a policy

Proximal policy optimization (PPO) computes an efficient surrogate objective to limit the change in the policy when executing multiple updates:

$$\mathcal{L}(\phi) = -\min \begin{pmatrix} \rho(s^t, a^t) Adv(s^t, a^t), \\ \text{clip}\left(\rho(s^t, a^t), 1-\epsilon, 1+\epsilon\right) Adv(s^t, a^t) \end{pmatrix}$$

PPO Surrogate Objective

$$\mathcal{L}(\phi) = -\min \left(\begin{array}{l} \rho(s^t, a^t) Adv(s^t, a^t), \\ \text{clip}\left(\rho(s^t, a^t), 1-\epsilon, 1+\epsilon\right) Adv(s^t, a^t) \end{array} \right)$$

- ρ represents the importance sampling ratio $\rho(s, a) = \frac{\pi(a|s;\phi)}{\pi_\beta(a|s)}$
- π_β represents the behavior policy followed to select action a^t in state s^t
- ϵ is a hyperparameter that determines the allowed change of the policy

PPO Surrogate Objective

$$\mathcal{L}(\phi) = -\min \left(\begin{array}{l} \rho(s^t, a^t) Adv(s^t, a^t), \\ \text{clip}\left(\rho(s^t, a^t), 1-\epsilon, 1+\epsilon\right) Adv(s^t, a^t) \end{array} \right)$$

- ρ represents the importance sampling ratio $\rho(s, a) = \frac{\pi(a|s;\phi)}{\pi_\beta(a|s)}$
- π_β represents the behavior policy followed to select action a^t in state s^t
- ϵ is a hyperparameter that determines the allowed change of the policy

Importance sampling ratios serve multiple purposes:

- Correct for differences in data distributions of π_β and π
- Measure of divergence between π_β and π $\rightarrow$ can be clipped to limit divergence

PPO Pseudocode

Algorithm Simplified proximal policy optimization (PPO)

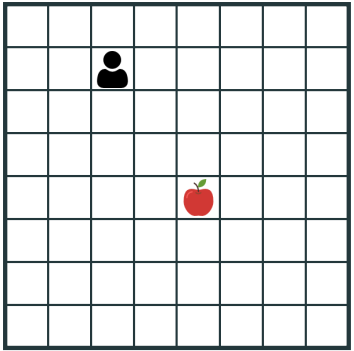
```
Initialize actor network π with random parameters φ
Initialize critic network V with random parameters θ
for every episode do
    for time step t = 0, 1, 2, ... do
        Observe current state s^t
        Sample action a^t ~ π(· | s^t; φ)
        Apply action a^t; observe reward r^t and next state s^{t+1}
        π_β(a^t | s^t) ← π(a^t | s^t; φ)
        for epoch e = 1, ..., N_e do
            ρ(s^t, a^t) ← π(a^t | s^t; φ) ÷ π_β(a^t | s^t)
            if s^{t+1} is terminal then
                Advantage Adv(s^t, a^t) ← r^t − V(s^t; θ)
                Critic target y^t ← r^t
            else
                Advantage Adv(s^t, a^t) ← r^t + γ V(s^{t+1}; θ) − V(s^t; θ)
                Critic target y^t ← r^t + γ V(s^{t+1}; θ)
            Actor loss L(φ) ← −min ( ρ(s^t, a^t) Adv(s^t, a^t),
                                     clip (ρ(s^t, a^t), 1 − ε, 1 + ε) Adv(s^t, a^t) )
            Critic loss L(θ) ← (y^t − V(s^t; θ))^2
            Update parameters φ by minimising the actor loss L(φ)
            Update parameters θ by minimising the critic loss L(θ)
```

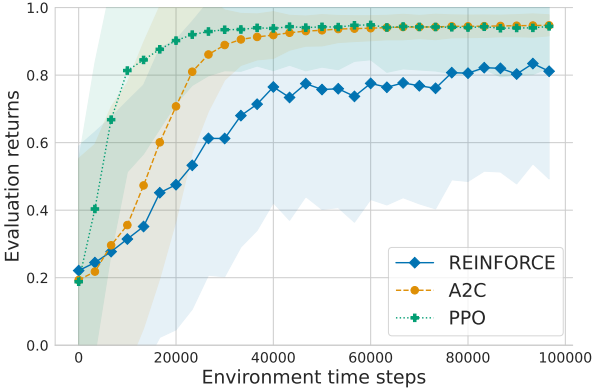
- PPO executes multiple epochs of updates!
- For first epoch, $\pi = \pi_\beta$

  $\Rightarrow \rho = 1$
- After first epoch, $\pi \neq \pi_\beta$

  $\Rightarrow$ needs ρ to correct for off-policy data

Policy Gradient Algorithms in Practice





PPO in Reinforcement Learning from Human Feedback (RLHF)

Pipeline

1. Supervised Fine-Tuning (SFT) on human demonstrations
2. Reward Model (RM) trained from human pairwise rankings
3. Policy optimization using PPO

We also experiment with mixing the pretraining gradients into the PPO gradients, in order to fix the performance regressions on public NLP datasets. We call these models "PPO-ptx." We maximize the following combined objective function in RL training:

$$\begin{aligned}\text{objective}(\phi) = & E_{(x,y)\sim D_{\pi_\phi^{\mathrm{RL}}}}\left[r_\theta(x,y) - \beta \log\left(\pi_\phi^{\mathrm{RL}}(y \mid x)/\pi^{\mathrm{SFT}}(y \mid x)\right)\right] + \\ & \gamma E_{x\sim D_{\text{pretrain}}}\left[\log(\pi_\phi^{\mathrm{RL}}(x))\right]\end{aligned} \tag{2}$$

where π_ϕ^{RL} is the learned RL policy, π^{SFT} is the supervised trained model, and D_{pretrain} is the pretraining distribution. The KL reward coefficient, β, and the pretraining loss coefficient, γ, control the strength of the KL penalty and pretraining gradients respectively. For "PPO" models, γ is set to 0. Unless otherwise specified, in this paper InstructGPT refers to the PPO-ptx models.

Training language models to follow instructions with human feedback (**link**)

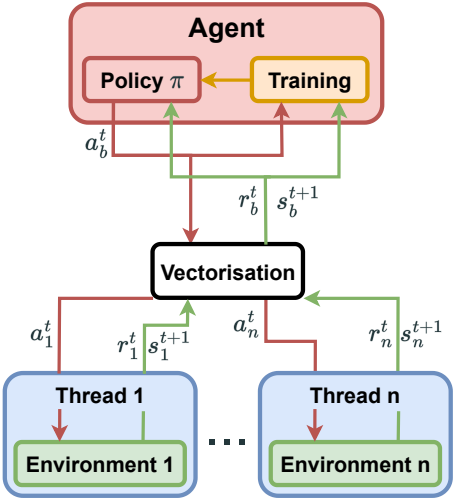
Concurrent Training

Concurrent Training of Policies

Policy gradient algorithms rely on on-policy data, which raises the question of how to deal with correlation in the data and how to increase sample efficiency.

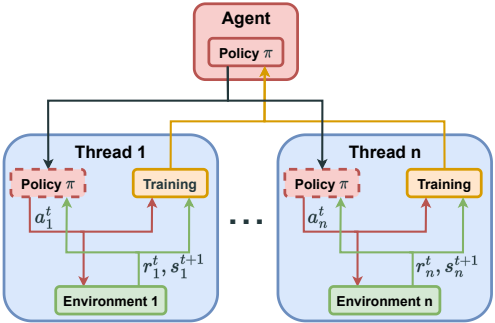
- Concurrent training of policies speeds up training by getting more samples (in parallel), leading to better gradients and breaking of correlation of data
- There are many ways to achieve this, but there are two simple methods commonly used, **synchronous training** and **asynchronous training**

Synchronous Training



- Initiates separate instances of the environment in separate threads
- At each timestep, the agent receives a batch of states and rewards from each thread
- The agent then independently chooses an action for each thread/environment
- Aggregate gradients across batch of experiences $\rightarrow$ more stable and efficient optimization

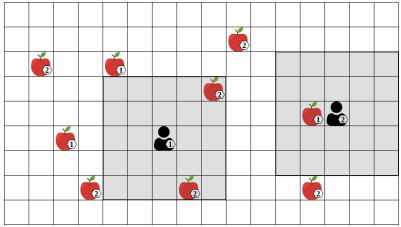
Asynchronous Training



- Asynchronous training parallelizes the optimization of the agent.
- Each thread separately computes the loss and gradients and optimizes the parameters of the agent's network
- Once gradients are computed, the central agent's network is updated
- Asynchronous training is particularly effective if multiple accelerators (e.g. GPUs) are available

Observation, States, and Histories in Practice

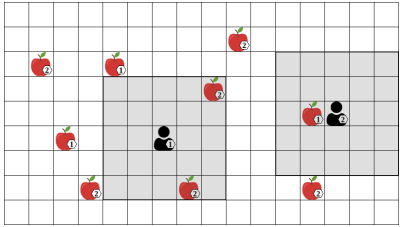
We have thus far only considered algorithms that condition on the entire states of the environment, but in practice we often have **partial observability**.



We want to condition value functions and policies on observation history $h^t = (o^0, ..., o^t)$

Observation, States, and Histories in Practice

We have thus far only considered algorithms that condition on the entire states of the environment, but in practice we often have **partial observability**.

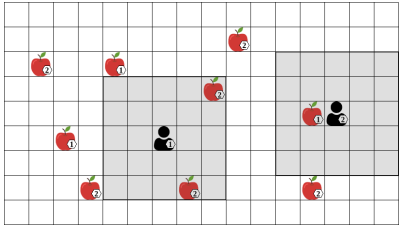


- Feedforward NN assume constant input size, this would require zero-padding vectors to the maximum episode length
- Zero-padding requires knowledge about maximum episode length and results in high-dimensional and sparse inputs

We want to condition value functions and policies on observation history $h^t = (o^0, ..., o^t)$

Observation, States, and Histories in Practice

We have thus far only considered algorithms that condition on the entire states of the environment, but in practice we often have **partial observability**.



We want to condition value functions and policies on observation history $h^t = (o^0, ..., o^t)$

- Feedforward NN assume constant input size, this would require zero-padding vectors to the maximum episode length
- Zero-padding requires knowledge about maximum episode length and results in high-dimensional and sparse inputs
- To avoid this we can use RNNs that process sequences of observations with one observation at a time while maintaining the previous history in the hidden state

Summary

We covered:

- Deep Q-learning
- Moving-target problem and correlations of consecutive experiences
- Policy gradient algorithms
- Concurrent training of policies
- Observation, states, and histories under partial observability

Next we'll cover:

- Games and MARL basics

Interactive Coding and Learning Session

Deep Reinforcement Learning – Textbook References

References

- *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. See chapters 7 and 8. A possible future "bible" for multi-agent RL. **link**
- *Reinforcement Learning: An Introduction*. See chapters 9 and 13. The "bible" of sigle-agent RL. **link**

Computational Resources

- **marl-book code**
- **sarl-book code**

Deep Reinforcement Learning – Foundational Algorithms

Original Papers

- DQN: *Human-level control through deep reinforcement learning*. **link**
- DDQN: *Deep Reinforcement Learning with Double Q-learning*. **link**
- REINFORCE: *Simple Stat. Gradient-Following Algs. for Connectionist Reinforcement Learning*. **link**
- A2C: *Asynchronous Methods for Deep Reinforcement Learning*. **link**
- PPO: *Proximal Policy Optimization Algorithms*. **link**
- DDPG: *Continuous Control with Deep Reinforcement Learning*. **link**
- SAC: *Soft Actor-Critic: Off-Policy Max. Ent. Deep Reinforcement Learning with a Stoch. Actor*. **link**

Computational Resources (Python)

- **Gymnasium**
- **Stable-Baselines3**
- **PyTorch RL**

Computational Resources (Julia)

- **POMDPs.jl**
- **Crux.jl**
- **ReinforcementLearning.jl**

Deep Reinforcement Learning – Application in Economics

References

- *Natural-gas storage modelling by deep reinforcement learning*, ICAIF 2025. **link**

Computational Resources

- **Python code**
- **Julia code**